

Bagaimana *Physics-Informed Neural Networks* (PINNs) bekerja?

Fajar Adie Santosa (@debug.stack)

April 27, 2026

1 Pengertian PINNs

Physics-Informed Neural Networks (PINNs) merupakan gabungan Neural Networks dengan menggabungkan hukum fisika yang di deskripsikan dengan persamaan diferensial ke dalam fungsi loss untuk proses pelatihan ke arah solusi yang lebih akurat dengan mendasari hukum fisika yang berlaku [1]. PINNs bisa digunakan untuk :

- Pendekatan solusi untuk persamaan diferensial biasa (PDB) dan persamaan diferensial parsial (PDP)
- Menyelesaikan *inverse problems*, seperti estimasi parameter model dari data yang terbatas

2 Pendahuluan

Pada penjelasan ini, kita akan membahas bagaimana Physics-Informed Neural Networks (PINNs) bekerja dengan studi kasus *damped harmonic oscillator* yang diadaptasi dari tutorial [link tutorial](#). Tujuan utama pada penjelasan ini yakni:

1. Menulis kode PINNs dari scratch (dengan bantuan *toolbox deep learning MATLAB*)
2. Memahami tipe perbedaan antara PINNs dengan Neural Networks biasa

3. Memahami lebih detail bagaimana PINNs selama training dan bagaimana PINNs bisa improve performa (konvergen) dengan cepat

3 Problem Overview

3.1 Oscillator Harmonik Teredam

Kita akan menyelesaikan masalah oscilator harmonik teredam dengan menggunakan PINNs:

Ini merupakan masalah fisika, yang dimana perpindahan $u(t)$ merupakan fungsi dari waktu oscilator yang bisa kita deskripsikan dengan persamaan diferensial :

$$m \frac{d^2 u}{dt^2} + \mu \frac{du}{dt} + ku = 0 \quad (1)$$

dimana m adalah massa, μ adalah koefisien redaman, dan k adalah konstanta pegas. Kita akan fokus menyelesaikan masalah ini di "under-damped state", yang dimana osilasi menjadi pelan seiring waktu (seperti gambar di atas).

Secara matematis, keadaan tersebut bisa ditulis:

$$\delta < \omega_o, \text{ dimana } \delta = \frac{\mu}{2m}, \omega_o = \sqrt{\frac{k}{m}} \quad (2)$$

Terlebih lagi, kita memikirkan dengan mengikuti kondisi awal dengan sistem:

$$u(t=0) = 1, \frac{du}{dt}(t=0) = 0. \quad (3)$$

Untuk kasus ini, kita menggunakan exact solusi yang diberikan oleh:

$$u(t) = e^{-\delta t} (2A \cos(\phi + \omega t)), \quad \omega = \sqrt{\omega_o^2 - \delta^2} \quad (4)$$

Untuk lebih detail matematis terkait osilator harmonik teredam, dapat dilihat melalui [5].

3.2 Workflow overview

Untuk menyelesaikan masalah oscilator harmonik teredam dengan menggunakan PINNs, kita akan mengikuti workflow berikut:

1. Membuat Neural Network
2. Melatih Neural Network dengan optimasi L-BFGS dan Levenberg-Marquadt
3. Membuat loss function yang menggabungkan persamaan diferensial dan kondisi awal menggunakan ansatz (hard constraint) untuk memastikan bahwa solusi yang dihasilkan oleh PINNs memenuhi kondisi awal
4. Membuat gradien dari loss function dengan menggunakan automatic differentiation untuk menghitung turunan yang diperlukan dalam persamaan diferensial (seperti $\frac{du}{dt}$ dan $\frac{d^2u}{dt^2}$)
5. Melakukan optimasi untuk meminimalkan loss function dengan menggunakan algoritma optimasi seperti L-BFGS atau Levenberg-Marquadt untuk memperbarui parameter dari PINNs
6. Mengevaluasi performa PINNs dengan membandingkan solusi yang dihasilkan dengan solusi eksak dan menghitung error untuk menilai akurasi dari PINNs

Untuk rumus ansatz yang digunakan yakni $u_{PINN} = 1 + t^2 \cdot \mathcal{N}(t; \theta)$ yang dimana $\mathcal{N}(t; \theta)$ adalah neural network dengan parameter θ . Kemudian, kita akan mendefinisikan fungsi loss (*Loss function*) untuk pelatihan PINN yang terdiri:

$$\mathcal{L} = \tag{5}$$

References

- [1] MathWorks, *MATLAB Documentation*, <https://www.mathworks.com/discovery/physics-informed-neural-networks.html>
- [2] M. Raissi, P. Perdikaris, dan G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, vol. 378, hal. 686–707, 2019. <https://doi.org/10.1016/j.jcp.2018.10.045>
- [3] I. E. Lagaris, A. Likas, dan D. I. Fotiadis, *Artificial neural networks for solving ordinary and partial differential equations*, IEEE Transactions on Neural Networks, vol. 9, no. 5, hal. 987–1000, 1998. <https://doi.org/10.1109/72.712178>

- [4] MathWorks, *Solve Partial Differential Equations with L-BFGS Method and Deep Learning*, MATLAB Documentation, 2024. <https://www.mathworks.com/help/deeplearning/ug/solve-partial-differential-equations-with-lbfgs-method-and-deep-learning.html>
- [5] B. Umläuft, *The Harmonic Oscillator*, beltoforion.de, diakses 2024. https://beltoforion.de/en/harmonic_oscillator/